

Instituto Tecnológico de Costa Rica

Curso: Diseño de Sistemas Digitales EL-3310

Profesor: Ing. Michael González Vega

Evaluación: Proyecto Final

“Diseño e implementación de un procesador RISC-V uniciclo”

Valor: 7%

Fecha límite de entrega:

26 de Octubre de 2026, 11:59 P.M.

Fecha de defensa: Será comunicada por el profesor.

Modalidad: En grupos

Capítulo 1 — Los cimientos

“Antes de construir el procesador, debemos construir y verificar sus cimientos.”

1. El reto

Un procesador no es una única pieza como se ha analizado a lo largo del semestre. Está formado por diferentes componentes que deben trabajar de manera coordinada. En este proyecto, cada equipo asumirá el papel de un pequeño equipo de ingeniería encargado de diseñar, implementar y verificar un procesador RISC-V de un ciclo.

El proyecto completo se desarrollará en tres capítulos. En esta primera etapa todavía no se construirá el procesador completo. El objetivo es desarrollar y verificar los componentes fundamentales que posteriormente formarán parte del datapath.

Al finalizar esta etapa, el equipo deberá demostrar que sus componentes funcionan de acuerdo con la especificación y que están preparados para integrarse en las siguientes etapas.

2. Objetivo de la primera entrega

Diseñar, implementar y verificar en SystemVerilog los componentes fundamentales de un procesador RISC-V, utilizando una metodología modular y pruebas funcionales que permitan demostrar el correcto funcionamiento de cada componente.

Al finalizar esta etapa, el estudiante deberá ser capaz de:

- Implementar módulos digitales utilizando SystemVerilog.
- Diseñar módulos independientes que puedan integrarse posteriormente.
- Implementar una ALU básica.
- Implementar un banco de registros.
- Implementar la generación de inmediatos.
- Implementar multiplexores para el datapath.
- Diseñar testbenches básicos.
- Analizar resultados de simulación.
- Identificar y corregir errores.
- Documentar las pruebas realizadas.

3. Alcance técnico

Los siguientes componentes son obligatorios:

Componente	Módulo sugerido	Propósito
ALU	alu.sv	Operaciones aritméticas y lógicas básicas.
Banco de registros	register_file.sv	Lectura y escritura de registros RISC-V.
Generador de inmediatos	imm_gen.sv	Construcción y extensión de inmediatos.
Multiplexor	mux.sv	Selección entre diferentes fuentes de datos.
Testbenches	*_tb.sv	Verificación funcional de cada módulo.

4.1 ALU

La ALU deberá trabajar con operandos de 32 bits y soportar, como mínimo:

Operación	Código interno
ADD	0000
SUB	0001
AND	0010
OR	0011

Además, deberá generar una señal Zero que sea 1 cuando el resultado sea igual a cero.

4.2 Banco de registros

Debe implementar 32 registros de 32 bits, dos puertos de lectura y un puerto de escritura. La escritura estará controlada mediante RegWrite.

Regla obligatoria: el registro x0 debe permanecer siempre en cero. Una escritura dirigida a x0 no debe modificar su valor.

4.3 Generador de inmediatos

En esta primera entrega se deberán implementar, como mínimo, los formatos de inmediato necesarios para trabajar con instrucciones I-type y S-type. En ambos casos, el inmediato deberá extenderse correctamente a 32 bits mediante extensión de signo.

Para instrucciones I-type:

El inmediato de 12 bits se encuentra en:

- $\text{imm}[11:0] = \text{instr}[31:20]$

El bit $\text{instr}[31]$ corresponde al bit de signo y deberá utilizarse para realizar la extensión de signo a 32 bits.

Por lo tanto:

- $\text{ImmExt} = \{\{20\{\text{instr}[31]\}\}, \text{instr}[31:20]\}$

Para instrucciones S-type:

El inmediato se encuentra dividido en dos campos dentro de la instrucción:

- $\text{imm}[11:5] = \text{instr}[31:25]$
- $\text{imm}[4:0] = \text{instr}[11:7]$

Al igual que en I-type, $\text{instr}[31]$ corresponde al bit de signo. El inmediato extendido a 32 bits puede representarse como:

- $\text{ImmExt} = \{\{20\{\text{instr}[31]\}\}, \text{instr}[31:25], \text{instr}[11:7]\}$

El módulo deberá seleccionar y generar correctamente el inmediato de acuerdo con el formato de instrucción indicado.

- $\text{ImmSrc} = 00 \rightarrow \text{I-type}$
- $\text{ImmSrc} = 01 \rightarrow \text{S-type}$

4.4 Multiplexor

Se deberá implementar un multiplexor 2:1 parametrizable o equivalente, diseñado para ser reutilizado posteriormente en el datapath.

5. Interfaz de referencia

Interfaces de referencia para los diferentes módulos y testbenches son proporcionados en el archivo .zip que contiene también este documento.

6. Verificación

No basta con entregar el código. Cada componente debe estar acompañado por evidencia de que funciona correctamente. Para ello, cada módulo deberá contar con un testbench.

Como mínimo, se deberán verificar:

Módulo	Pruebas mínimas sugeridas
ALU	ADD, SUB, AND, OR, resultado cero y al menos un caso con resultado negativo.
Register File	Escritura/lectura, dos lecturas y comportamiento de x0.
Immediate Generator	Al menos dos casos I-type y dos casos S-type, incluyendo extensión de signo.
MUX	Selección de cada una de las dos entradas.

Los casos de prueba deberán estar claramente identificados y los resultados deberán poder relacionarse con los valores esperados.

Los testbenches deberán determinar de forma automática o claramente identificable si cada prueba cumple con el resultado esperado. No será suficiente únicamente con generar señales y observarlas visualmente.

7. Matriz de pruebas

El equipo deberá incluir una matriz de pruebas. Se recomienda el siguiente formato:

ID	Módulo	Entrada / condición	Esperado	Obtenido	Estado
ALU-01	ALU	5 + 3	8	8	PASS
ALU-02	ALU	5 - 3	2	2	PASS
ALU-03	ALU	5 - 5	0 / Zero=1	0 / Zero=1	PASS
RF-01	Register File	Write x5=25; read x5	25	25	PASS
RF-02	Register File	Write x0=100; read x0	0	0	PASS

8. Roles del equipo

Cada integrante tendrá un rol principal. Los roles representan responsabilidades, no una división absoluta del trabajo: todos los integrantes deben conocer el funcionamiento general del proyecto y participar técnicamente.

Rol	Responsabilidades principales
Team Lead	Organización, distribución de tareas, seguimiento, integración y coordinación de la entrega.
Hardware Developer	Implementación de módulos SystemVerilog, depuración e integración. En equipos de 5 habrá dos developers.
Verification Engineer	Diseño de testbenches, casos de prueba, análisis de resultados e identificación de errores.
Documentation & Configuration Engineer	Documentación, diagramas, estructura del proyecto, control de cambios y consolidación de evidencias.

El rol asignado a cada estudiante deberá indicarse en el README.md y en el informe técnico.

9. Entregables

La entrega estará compuesta por cuatro elementos:

1. Código fuente

Se deberá entregar un archivo .zip que contenga todos los archivos SystemVerilog (.sv) desarrollados durante esta etapa, incluyendo los módulos y sus respectivos testbenches.

Los archivos deberán mantener la estructura, nomenclatura y encabezados definidos para el proyecto.

La estructura solicitada es:

```
RISC-V-CPU/
├── src/
│   ├── alu.sv
│   ├── register_file.sv
│   ├── imm_gen.sv
│   └── mux.sv
├── testbench/
│   ├── alu_tb.sv
│   ├── register_file_tb.sv
│   ├── imm_gen_tb.sv
│   └── mux_tb.sv
├── docs/
│   └── GrupoXXXXXXXXXinforme.pdf
└── README.md
```

2. Simulación en EDA Playground

Cada módulo deberá contar con una simulación funcional en EDA Playground que permita verificar su comportamiento mediante el testbench desarrollado por el grupo.

Para esta primera entrega se deberán proporcionar cuatro enlaces de EDA Playground, correspondientes a:

- ALU
- Register File
- Immediate Generator
- MUX

Cada Playground deberá contener el módulo correspondiente y su testbench. Los enlaces deberán permitir al profesor abrir el proyecto y ejecutar directamente la simulación. Los cuatro enlaces de EDA Playground deberán incluirse en el archivo README.md, claramente identificados por módulo.

Los resultados obtenidos deberán ser consistentes con los casos de prueba documentados en la matriz de pruebas.

3. Informe técnico corto

Se deberá presentar un informe técnico de máximo 10 páginas, que incluya:

- Descripción general de los componentes desarrollados.
- Decisiones principales de diseño.
- Estrategia utilizada para verificar cada módulo.
- Resultados obtenidos.
- Principales problemas encontrados durante el desarrollo y cómo fueron solucionados.
- Conclusiones de la etapa.

4. Matriz de pruebas

Se deberá presentar una matriz que documente los casos de prueba ejecutados sobre cada módulo.

Como mínimo, cada caso deberá indicar:

ID | Módulo | Entradas/Condición | Resultado esperado | Resultado obtenido | Estado

El estado de cada prueba deberá indicarse como PASS o FAIL según corresponda.

10. Defensa técnica

La entrega incluirá una defensa técnica de aproximadamente 15 minutos por equipo. La defensa busca comprobar tanto el funcionamiento del proyecto como la comprensión individual.

Actividad	Tiempo aproximado
Presentación del equipo y organización del trabajo	3 min
Demostración de simulaciones y pruebas	7 min
Preguntas técnicas individuales	5 min

Durante la defensa, el profesor podrá solicitar cambios sencillos en las entradas de una prueba o pedir a un integrante que explique una parte específica del código. Las preguntas podrán dirigirse a cualquier integrante del equipo.

11. Evaluación — 7% del curso

La calificación considerará el funcionamiento, la calidad de la verificación, la documentación y el trabajo del equipo.

Criterio	Valor	Descripción
Implementación SystemVerilog	2.25%	Los módulos solicitados se encuentran implementados correctamente, presentan una estructura clara, respetan las interfaces definidas y siguen las convenciones establecidas para el proyecto.
Funcionamiento de los componentes	1.5%	La ALU, Register File, Immediate Generator y MUX presentan el comportamiento esperado de acuerdo con las especificaciones de esta etapa.
Verificación y testbenches	1.5%	Se desarrollan casos de prueba adecuados para verificar el comportamiento de los módulos. Los testbenches permiten identificar claramente los resultados obtenidos y contemplan condiciones relevantes de funcionamiento.
Documentación y matriz de pruebas	0.75%	El informe presenta de forma clara el diseño realizado, decisiones tomadas, resultados, problemas

		encontrados y conclusiones. La matriz de pruebas mantiene correspondencia con las pruebas ejecutadas.
Organización, autoría y defensa técnica	1%	El grupo mantiene una adecuada distribución de responsabilidades y trazabilidad de modificaciones. Durante la defensa, los integrantes demuestran comprensión del código, documentación y pruebas entregadas, y son capaces de explicar o realizar modificaciones menores sobre su trabajo.
TOTAL	7.0%	

12. Consideraciones sobre la defensa técnica

La evaluación no se limitará únicamente al funcionamiento final de los módulos. Durante la defensa, el profesor podrá seleccionar aleatoriamente integrantes del grupo y solicitarles explicar cualquier parte relevante del proyecto.

Las preguntas podrán incluir aspectos relacionados con:

- Funcionamiento de un módulo.
- Interpretación de señales de entrada y salida.
- Decisiones tomadas durante la implementación.
- Funcionamiento de un testbench.
- Interpretación de un caso de prueba.
- Explicación de fragmentos del código o documentación.
- Realización de una modificación menor.
- Análisis de un caso o entrada diferente a los utilizados originalmente.

No es necesario que todos los integrantes hayan desarrollado directamente cada archivo; sin embargo, todos deberán conocer y comprender la arquitectura general y ser capaces de explicar el trabajo presentado por el grupo.

La calificación del proyecto podrá variar entre integrantes de un mismo grupo de acuerdo con el desempeño individual demostrado durante la defensa técnica. Esta valoración se reflejará principalmente en el criterio “Organización, autoría y defensa técnica”.

13. Uso de herramientas de Inteligencia Artificial y autoría del trabajo

Las herramientas de Inteligencia Artificial podrán utilizarse como apoyo para el aprendizaje, consulta o resolución de dudas. Sin embargo, el contenido entregado deberá ser elaborado, comprendido y validado por los integrantes del grupo.

No se permitirá la incorporación directa de código, explicaciones, análisis, conclusiones u otros contenidos generados por herramientas de Inteligencia Artificial sin que estos hayan sido revisados, comprendidos y adaptados por el grupo.

Durante la revisión del proyecto, el profesor podrá solicitar a cualquier integrante:

- Explicar fragmentos del código o del informe entregado.
- Justificar decisiones de diseño e implementación.
- Explicar el funcionamiento de los testbenches y casos de prueba.

- Realizar modificaciones menores al código.
- Analizar resultados o comportamientos no incluidos previamente en la entrega.

El contenido que presente inconsistencias con el resto del trabajo o cuya autoría o comprensión genere dudas podrá ser revisado con mayor detalle durante la defensa.

Todo integrante deberá ser capaz de explicar y defender el contenido presentado por su grupo. La incapacidad de explicar o justificar partes relevantes de la entrega podrá afectar la calificación correspondiente, de acuerdo con la rúbrica del proyecto.

14. ¿Qué sigue?

Esta entrega constituye la base del procesador. Los módulos desarrollados aquí serán reutilizados en la siguiente etapa, donde el equipo comenzará a integrar el datapath.

Capítulo 1 — Los cimientos → Capítulo 2 — El datapath cobra vida → Capítulo 3 — Nuestro procesador

El objetivo de esta primera misión es sencillo: construir componentes en los que podamos confiar.